

《编译原理》实验报告

姓名：李经雄

学院：计算机科学与技术学院

专业：计算机科学与技术

邮箱：3220105346@zju.edu.cn

报告日期：2024. 4. 9

Lab3 Zero IR 中间代码生成

一 实验内容

1.1 IR 结构定义

在ir.hpp中定义需要用到的所有IR代码，按照实验文档之中的zero IR的代码定义——实现即可。我实现了LoadImm Assign Binary Unary Label If Goto Function Call Arg Return Param Dec Global Store Load AddressOf 等类。

1.2 翻译 IR

逐步的递归翻译IR是本次实验的难点所在。我一共实现了下面这些翻译函数，其中值得注意的是对表达式的翻译由于需要将返回值赋值给place所以我们将place作为一个额外的参数传递，对条件语句的翻译由于需要支持短路操作所以增加了true_label和false_label变量。

```
1  IR::Code translateCond(AST::NodePtr cond, const std::string &true_label, const
    std::string &false_label);
2
3  IR::Code translatePrimaryExp(AST::PrimaryExpPtr node, const std::string &place = "");
4  IR::Code translateLVal(AST::LValPtr node, const std::string &place = "");
5  IR::Code translateBinaryExp(AST::BinaryExpPtr node,
6                               const std::string &place = "");
7  IR::Code translateUnaryExp(AST::UnaryExpPtr node,
8                              const std::string &place = "");
9  IR::Code translateFuncCall(AST::FuncCallPtr node,
10                             const std::string &place = "");
11 IR::Code translateIntConst(AST::IntConstPtr node,
12                             const std::string &place = "");
```

二 实现亮点

2.1 全局数组与局部数组定义

这里的数组初始化类似lab2的语义检查，但是需要把初始化列表补全并转化为相应的中间代码。这里我重新参考lab2完成了两个辅助函数。对于全局数组来说实现比较简单，只需要将初值列表变成一个int型的向量传回即可，而对于局部数组来说，因为需要对每一个元素的赋值都单独生成一条指令，所以我们需要将基地址place和已填充元素filledCount都传入函数用以计算偏移量。

由于全局数组没有赋初值的话默认全0，其效果等效于int a[10] = {};直接手动给他添加一个{}即可把这种情况归类到一般之中，而对于局部数组没有赋初值的情况我们什么都不做，元素的值是未定状态。

```
1  std::vector<int> fillupGlobalInitList(const std::vector<AST::InitValPtr>& sublist,
    const std::vector<int>& dims);
2  IR::Code fillupLocalInitList(const std::vector<AST::InitValPtr> &sublist, const
    std::vector<int> &dims, const std::string &place = "", int filledCount = 0);
```

函数内部的具体实现思路和lab2的语义检查大致相同，都是遇到{}就递归调用自身求取最大可填充的子数组，遇到Exp就生成具体的指令，此时元素的地址是place+(filledCount) * 4。当遇到需要补充0的时候，补0个数是对当前的数组来说totalCount-nowCount，而补0的地址却是place+(filledCount) * 4，这也说明了为什么需要传递place和filledCount参数。

```
1  for (auto child : sublist) {
2      auto initv = std::dynamic_pointer_cast<AST::InitVal>(child);
3      if (initv->kind == AST::InitVal::Kind::EXP) {
4          auto ir_exp = translateExp(initv->expr, temp);
5          std::move(ir_exp.begin(), ir_exp.end(), std::back_inserter(result));
6          result.push_back(IR::Store::create(place, filledCount * 4, temp));
7      } else {
8          auto subResult = fillupLocalInitList(sublist, subDims, place, filledCount);
9          std::move(subResult.begin(), subResult.end(), std::back_inserter(result));
10     }
11 }
12 if (nowCount < totalCount) {
13     for (int i = nowCount; i < totalCount; i++) {
14         auto temp = new_temp();
15         result.push_back(IR::LoadImm::create(temp, 0));
16         result.push_back(IR::Store::create(place, (filledCount) * 4, temp));
17     }
18 }
```

2.2 LVal 作函数实参

LVal在我们的实验中不是完全意义上的左值，而函数传递实参的时候LVal可以表达一整个数组的类型，这个时候关键是要计算出实际传递地址值相对于数组基地址的偏移量。首先一个需要明确的关键点在于node->dims存储的是LVal节点的维度信息，而node->symbol->type中记录的才是这个变量本身实际的维度信息。所以我们先通过node->symbol->type判断此时LVal本质上是不是数组。如果是数组的话，那么偏移量的计算就是下标值*对应子数组大小的总和，实现上用两层for循环，最后还需要比较是否已经求值到元素，如果是的话就直接用Load将元素地址变成值传递值即可，否则需要直接传递数组地址。

```
1  IR::Code IRTranslator::translateLVal(AST::LValPtr node, const std::string &place) {
2      IR::Code ir;
3      auto Type = std::dynamic_pointer_cast<ArrayType>(node->symbol->type);
4      if (Type == nullptr || Type->dims.empty()) {
5          ...
6      } else {
7          for (std::vector<int>::size_type i = 0; i < node->dims.size(); i++) {
8              ir.push_back(IR::LoadImm::create(offsetTemp, 1));
9              for (std::vector<int>::size_type j = i + 1; j < arrType->dims.size(); j++) {
10                 subArrSize *= arrType->dims[j];
11             }
12             auto exp_ir = translateExp(dimExp, idxTemp);
13             std::move(exp_ir.begin(), exp_ir.end(), std::back_inserter(ir));
14             ir.push_back(IR::Binary::create(offsetTemp, idxTemp, BinaryOp::Mul,
15 IR::imm_str(subArrSize * 4)));
16             ir.push_back(IR::Binary::create(baseName, baseName, BinaryOp::Add, offsetTemp));
17         }
18         if (!place.empty() && node->dims.size() == arrType->dims.size()) {
```

```

18     ir.push_back(IR::Load::create(place, baseName));
19 } else if(!place.empty()) {
20     ir.push_back(IR::Assign::create(place, baseName));
21 }
22 }
23 return ir;
24 }

```

2.3 条件表达式短路操作

在实验中，条件表达式Relop(> >= < <= == !=)六大关系运算符两侧的表达式都是Exp类型，即可调用translateExp求值的表达式类型，或者说bool类型的操作符只有&& || 和!。在实现translateCond时可能遇到下面情况。

- &&和||：需要进行短路操作。对左右两边递归调用translateCond，并在中间插入短路的标签，这也是单独实现translateCond最重要的目的
- !：只需要简单将两个标签调换一下，在调用translateCond即可。
- 六大关系运算符：分别对左右两边调用translateExp，再用IF x relop y GOTO L中间代码实现跳转
- 五大算数运算符：分别对左右两边调用translateExp，再用x = y binop z求出最后的值，最后用IF x relop y GOTO L将结果和0比较决定是否跳转
- 其他各种Exp类型：先递归调用translateExp，然后用IF x relop y GOTO L将结果和0比较决定是否跳转

这里展示的是三个最重要的&& || 和!处理过程。

```

1  IR::Code IRTranslator::translateCond(AST::NodePtr cond, const std::string &true_label,
2  const std::string &>false_label) {
3      if (auto n = std::dynamic_pointer_cast<AST::BinaryExp>(cond)) {
4          switch (n->op) {
5              case BinaryOp::LAnd:
6                  short_circuit = new_temp();
7                  left_ir = translateCond(n->left, short_circuit, false_label);
8                  std::move(left_ir.begin(), left_ir.end(), std::back_inserter(ir));
9                  ir.push_back(IR::Label::create(short_circuit));
10                 right_ir = translateCond(n->right, true_label, false_label);
11                 std::move(right_ir.begin(), right_ir.end(), std::back_inserter(ir));
12                 break;
13             case BinaryOp::LOr:
14                 short_circuit = new_temp();
15                 left_ir = translateCond(n->left, true_label, short_circuit);
16                 std::move(left_ir.begin(), left_ir.end(), std::back_inserter(ir));
17                 ir.push_back(IR::Label::create(short_circuit));
18                 right_ir = translateCond(n->right, true_label, false_label);
19                 std::move(right_ir.begin(), right_ir.end(), std::back_inserter(ir));
20                 break;
21             }
22         } else if (auto n = std::dynamic_pointer_cast<AST::UnaryExp>(cond)) {
23             switch (n->op) {
24                 case BinaryOp::Not:
25                     exp_ir = translateCond(n->exp, false_label, true_label);
26                     std::move(exp_ir.begin(), exp_ir.end(), std::back_inserter(ir));
27                     break;
28             }
29         }
30     }
31 }

```

三 AI工具使用

- ChatGPT：将框架代码交给ChatGPT分析，并让他解释代码框架的思路和我所需要完成的任务。描述待完成工作并让ChatGPT生成可以采取的实现形式供参考
- Copilot：辅助完成代码实现和注释

四 建议

谈不上建议，就写一点自己的感想吧。自己最开始没有理解临时变量的使用方法，以为所有的局部变量使用都需要用DEC声明，直到求助助教才弄明白这里的用法，感谢助教，后面也看见实验指导上更新了几个帮助理解的例子。

处理translateCond的时候，如果不明确关系运算符的两侧都是可求值的非布尔表达式的话其实还是挺麻烦的，如果这样就不知道应该在什么时候调用translateExp了。

Lval的node->dims和node->symbol->dims的差别是让我最直观地感受到符号表的用处的，如果没有符号表记录原来数组的维度信息，那当整个数组作为函数实参的时候想要计算偏移量就真的很困难了，想清楚这里的差别之后其实完成LVal的翻译就很快了。